

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Математико-механический факультет

Якубовская Анастасия Алексеевна

Оптимизация Лабораторной работы 1 с помощью добавления параллельности

Лабораторная работа

студента 1 курса дневного отделения

Санкт-Петербург

2025

Цель работы: ускорить выполнение операций обработки ВМР-изображений (поворот и применение гауссова фильтра) за счет внедрения параллельных вычислений с использованием OpenMP.

1. Основные изменения

В код были добавлены директивы OpenMP для параллелизации трех ключевых операций:

1. rotateRight() – поворот изображения вправо
2. rotateLeft() – поворот изображения влево
3. applyGaussianFilter() – применение гауссова размытия

Изменения в коде:

rotateRight():

```
#pragma omp parallel for
for (int y = 0; y < originalHeight; ++y)
{
    for (int x = 0; x < originalWidth; ++x)
    {
        int oldIndex = (y * originalWidth + x) * bytesPerPixel;
        int newIndex = ((x) * originalHeight + (originalHeight - y - 1)) * bytesPerPixel;

        for (int i = 0; i < bytesPerPixel; ++i)
        {
            rotatedPixels[newIndex + i] = pixels[oldIndex + i];
        }
    }
}
```

Рис1. Изменение rotateRight()

rotateLeft():

```
#pragma omp parallel for
for (int y = 0; y < originalHeight; ++y)
{
    for (int x = 0; x < originalWidth; ++x)
    {
        int oldIndex = (y * originalWidth + x) * bytesPerPixel;
        int newIndex = ((originalWidth - x - 1) * originalHeight + y) * bytesPerPixel;

        for (int i = 0; i < bytesPerPixel; ++i)
        {
            rotatedPixels[newIndex + i] = pixels[oldIndex + i];
        }
    }
}
```

Рис2. Изменение rotateLeft()

applyGaussianFilter():

```
#pragma omp parallel for
for (int y = kernelSize / 2; y < height - kernelSize / 2; ++y)
{
    for (int x = kernelSize / 2; x < width - kernelSize / 2; ++x)
    {
        float r = 0, g = 0, b = 0, a = 0;

        for (int ky = -kernelSize / 2; ky <= kernelSize / 2; ++ky)
        {
            for (int kx = -kernelSize / 2; kx <= kernelSize / 2; ++kx)
            {
                int pixel_x = x + kx;
                int pixel_y = y + ky;

                int pixelIndex = (pixel_y * width + pixel_x) * bytesPerPixel;

                if (pixelIndex >= 0 && static_cast<size_t>(pixelIndex + 3) < pixels.size())
                {
                    r += pixels[pixelIndex] * kernel[ky + kernelSize / 2][kx + kernelSize / 2];
                    g += pixels[pixelIndex + 1] * kernel[ky + kernelSize / 2][kx + kernelSize / 2];
                    b += pixels[pixelIndex + 2] * kernel[ky + kernelSize / 2][kx + kernelSize / 2];
                    if (bytesPerPixel == 4)
                        a += pixels[pixelIndex + 3] * kernel[ky + kernelSize / 2][kx + kernelSize / 2];
                }
            }
        }
    }
}
```

Рис3. Изменения applyGaussianFilter()

2. Изменения в Makefile:

В Makefile добавлен флаг -fopenmp для поддержки OpenMP:

```
CXXFLAGS = -I./include -std=c++17 -Wall -Wextra -Wpedantic -O3 -march=native -fopenmp
LDFLAGS = -fopenmp
```

Рис 4. Изменения Makefile

3. Измерение результатов

С помощью <chrono> было измерено время выполнения программы до и после оптимизации:

```
auto total_start = std::chrono::high_resolution_clock::now();

BMPImage image;

auto start = std::chrono::high_resolution_clock::now();
bool readSuccess = image.read("Airplane1.bmp");
auto end = std::chrono::high_resolution_clock::now();
std::cout << "Reading time: "
    << std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count()
    << " ms" << std::endl;
```

Рис 5. Пример измерения времени работы

4. Результаты измерения:

После оптимизации время работы моей программы снизилось в три раза (приблизительно)

До применения параллельности:

```
anastasya@anastasya-VirtualBox:~/Документы/LabWork/LabWork1$ ./BMPProcessing
Reading time: 43 ms
Left rotation time: 4 ms
Writing rotated left time: 3 ms
Double right rotation time: 7 ms
Writing double rotated right time: 3 ms
Reading rotated left time: 1 ms
Gaussian filter (left) time: 45 ms
Writing filtered left time: 3 ms
Reading rotated right second time: 1 ms
Gaussian filter (right second) time: 40 ms
Writing filtered right second time: 2 ms

Total execution time: 159 ms
```

Рис 6.Время работы до оптимизации

После применения:

```
anastasya@anastasya-VirtualBox:~/Документы/LabWork/LabWork1$ ./BMPProcessing
Reading time: 2 ms
Left rotation time: 2 ms
Writing rotated left time: 4 ms
Double right rotation time: 6 ms
Writing double rotated right time: 2 ms
Reading rotated left time: 1 ms
Gaussian filter (left) time: 15 ms
Writing filtered left time: 4 ms
Reading rotated right second time: 2 ms
Gaussian filter (right second) time: 10 ms
Writing filtered right second time: 3 ms

Total execution time: 57 ms
```

Рис 7. Время работы после оптимизации

